Solution: Swap Nodes in Pairs (easy)

Problem Statement

Given a singly linked list, swap every two adjacent nodes and return the head of the modified list.

If the total number of nodes in the list is odd, the last node remains in place. Every node in the linked list contains a single integer value.

Examples

1. **Input**: `[1, 2, 3, 4]`
   Output: `[2, 1, 4, 3]`
   Justification: Pairs (1,2) and (3,4) are swapped.
2. **Input**: `[7, 8, 9, 10, 11]`
   Output: `[8, 7, 10, 9, 11]`
   Justification: Pairs (7,8) and (9,10) are swapped. 11 remains in its place as it has no adjacent node to swap with.
3. **Input**: `[5, 6]`
   Output: `[6, 5]`
   Justification: The pair (5,6) is swapped.

Solution

To swap nodes in pairs in a linked list, start with a temporary node before the list's first node. This helps to easily handle swaps at the beginning. Then, go through the list, looking at two nodes at a time. To switch the position of two nodes, change the links between these nodes. After swapping a pair, move on to the next two nodes. Keep doing this until you reach the end of the list. When you're done, the list will have each pair of nodes swapped, but the order of any remaining single nodes will stay the same. Return the list starting from the node right after your temporary starting node.

Here is the step-by-step solution.

1. **Initialization**:
   - Initialize two pointers, `current` and `previous`. Set `current` to the head of the list and `previous` to `null`.
   - If the list has fewer than two nodes, return the head as it is.
2. **Swapping Nodes**:
   - For every pair of adjacent nodes, change the `next` pointer of the previous node to point to the second node of the pair, and change the `next` pointer of the first node of the pair to point to the node following the second node in the pair. Update the `next` pointer of the second node to point to the first node, effectively swapping them.
3. **Updating Pointers**:
   - After swapping, move the `current` pointer two steps forward to the next pair of nodes. Update the `previous` pointer to point to the node that was just swapped to its new position.
4. **Handling Edge Cases**:
   - If the list has an odd number of nodes, the last node will remain in its place since there is no adjacent node to swap with.

Algorithm Walkthrough:

Let's run our algorithm on the Example-1:

- **Step 1:** Initialize `current` to the head (1) and `previous` to `null`.
- **Step 2:** Swap nodes 1 and 2. Update `previous` to point to node 2.
- **Step 3:** Move `current` to node 3.
- **Step 4:** Swap nodes 3 and 4. Update `previous` to point to node 4.
- **Step 5:** Move `current` to `null`. The list is now `[2, 1, 4, 3]`.

Here is the visual representation of the algorithm:

create a **dummy node** and set its **next** to point to the **head**.
Also create a **previous node** variable to maintain the node previous to the current pair being swapped
Iinitialize it by setting it to **previous=dummy node**

create two variables, first and second to use for swapping
set first=head and second=head.next

Now set first.next=second.next

Now set second.next=first

Now set prev.next=second

Now set head=first.next and move previpus to the first node

Now again set first=head and second=head.next

Now set first.next=second.next and second.next=first

Now set prev.next=second

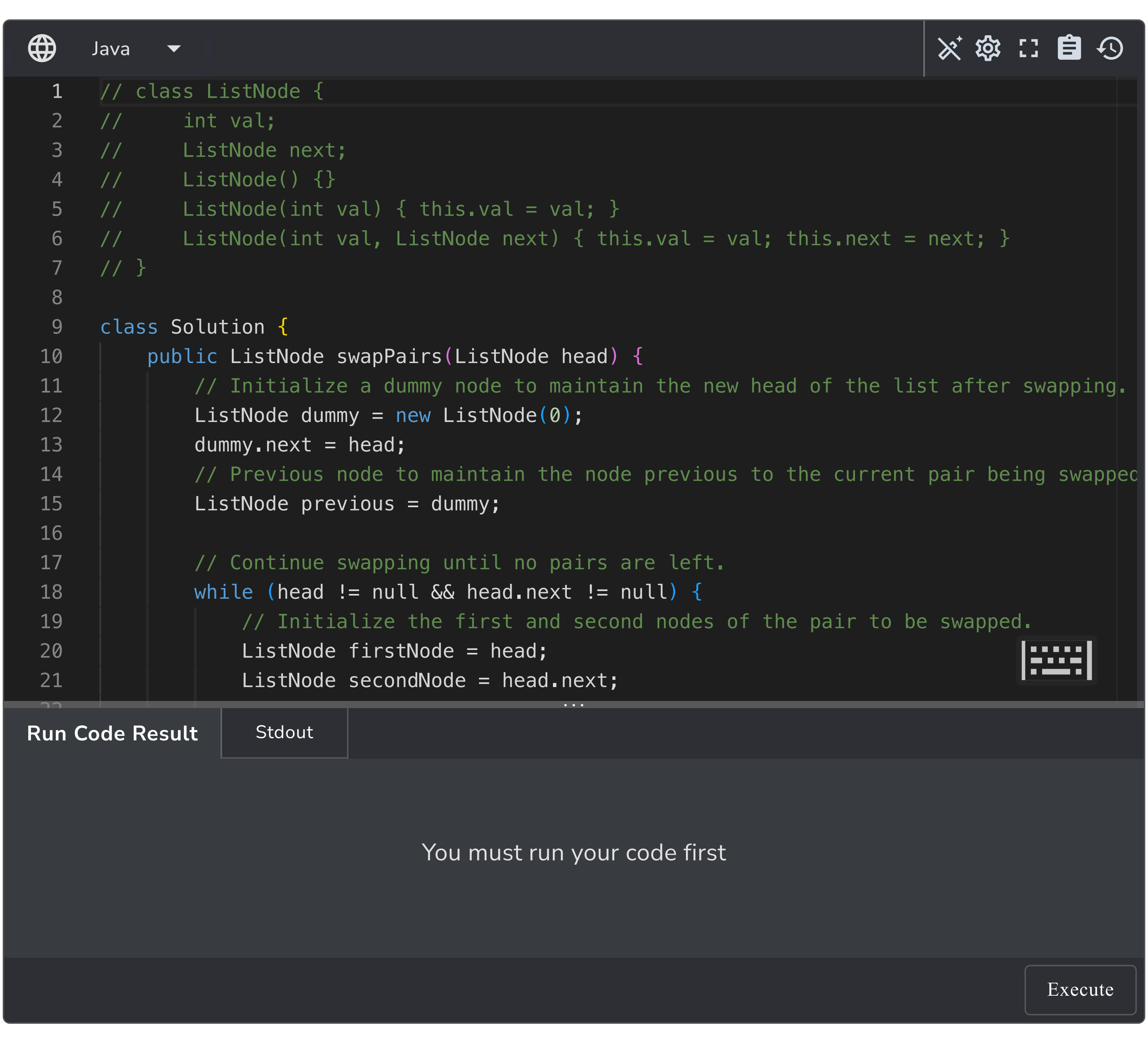
Now set head=first.next and previous=first

Since head is now NULL so loop will be terminated and we will **return dummy.next** as from there the final list starts

Swap Nodes in Pairs

Code

Here is the code for this algorithm:



```java
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode() {}
//     ListNode(int val) { this.val = val; }
//     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
// }

class Solution {
    public ListNode swapPairs(ListNode head) {
        // Initialize a dummy node to maintain the new head of the list after swapping.
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        // Previous node to maintain the node previous to the current pair being swappe
        ListNode previous = dummy;

        // Continue swapping until no pairs are left.
        while (head != null && head.next != null) {
            // Initialize the first and second nodes of the pair to be swapped.
            ListNode firstNode = head;
            ListNode secondNode = head.next;
```

You must run your code first

Time Complexity

- We traverse each node in the linked list exactly once.
- Each node involves a constant amount of work (swapping nodes).
- Since there are `n` nodes, the time complexity for traversing and processing the linked list is O(n).

Space Complexity

- We only use a constant amount of extra space to store pointers while performing the swaps. Thus, the space complexity is O(1).